

**Universidad Nacional de Ingeniería
Facultad de Ciencias**

Arquitectura de computadores

La Memoria

Prof.: Lic. César Martín Cruz Salazar
ccruz@uni.edu.pe

Locality

- *Locality*. Principio que establece que en un instante dado un programa accesa un espacio pequeño de su espacio de direcciones.
- *Locality en tiempo*. Si se accede a una dirección de memoria, es posible que vuelva a ser usado pronto.
- *Locality en espacio*. Si se accede a una dirección de memoria, es posible que las direcciones de memoria que se encuentran próximas a ella sean usadas pronto.
- Para tomar ventaja del *principio de locality*, se establece una ***jerarquía de memoria***.

Jerarquía de memoria

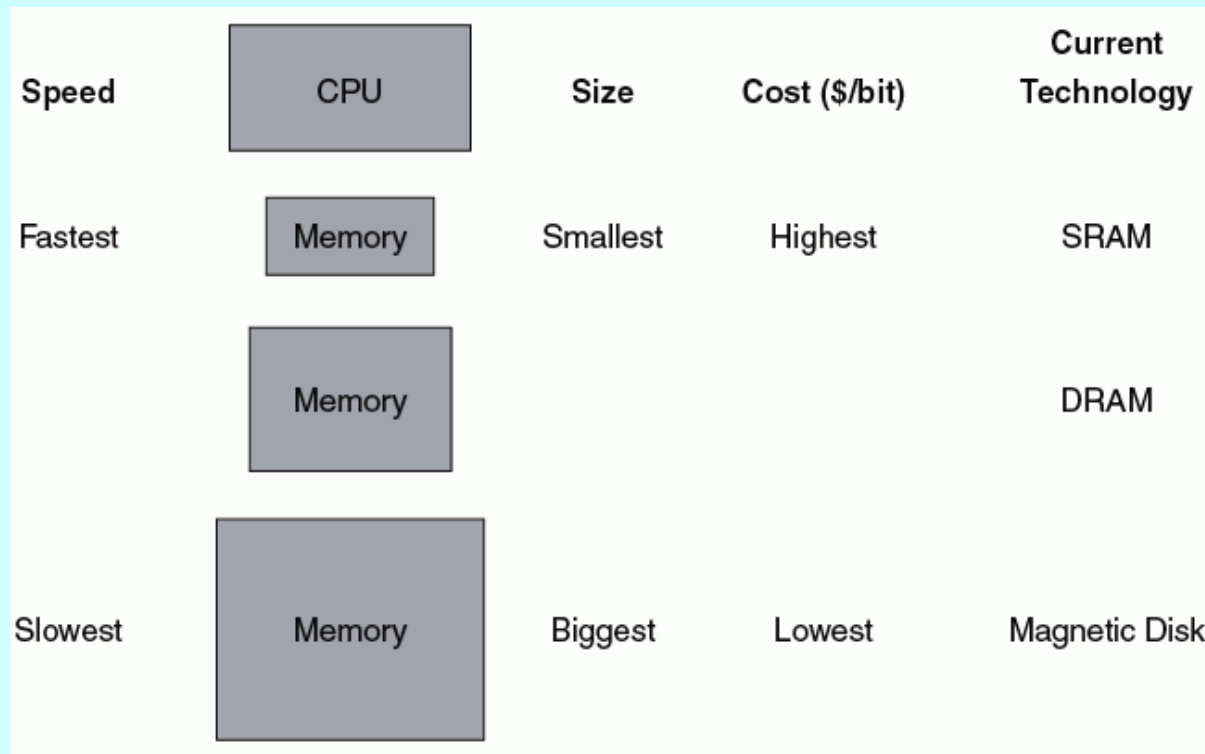
- Una jerarquía de memoria consiste de múltiples niveles de memoria con diferentes velocidades y tamaños.
- Las memorias mas rápidas son mas pequeñas y viceversa.

Memory technology	Typical access time	\$ per GB in 2004
SRAM	0.5–5 ns	\$4000–\$10,000
DRAM	50–70 ns	\$100–\$200
Magnetic disk	5,000,000–20,000,000 ns	\$0.50–\$2

- **SRAM** (*RAM estática*) se utiliza para el caché.
- **DRAM** (*RAM dinámica*) se utiliza para la memoria principal.

Jerarquía de la memoria

- En la jerarquía, la memoria mas rápida y pequeña está mas cerca de la CPU que la memoria mas lenta y grande.

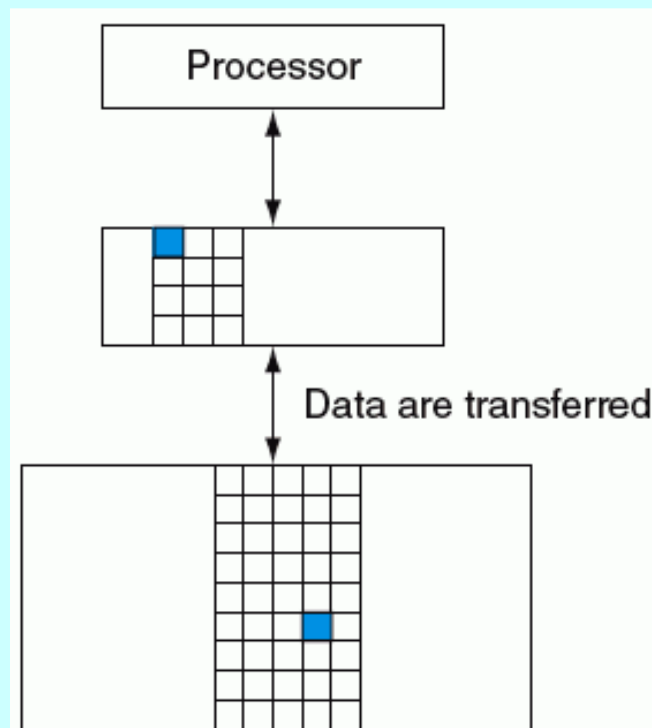


Jerarquía de la memoria

- Una jerarquía multinivel se ve como una jerarquía de 2 niveles porque en un instante dado los datos se copian solo entre niveles adyacentes.
- La memoria en el nivel superior es la mas cercana a la CPU y más pequeña y rápida que la memoria en el nivel inferior.

Bloques

- ***Bloque*** es la unidad mínima de información que puede estar o no estar presente en una jerarquía de 2 niveles.



Aciertos y fallos

- Un ***acierto*** (**hit**) se produce cuando los datos que busca la CPU están en el nivel superior.
- Si los datos no están, se produce un ***fallo*** (**miss**) y se buscan en el nivel inferior.

Medidas de rendimiento

- ***Tasa de aciertos*** (hit rate). Fracción de accesos de memoria encontrados en el nivel superior.
- ***Tasa de fallos*** (miss rate) = $(1 - \text{tasa de aciertos})$. Fracción de los accesos a memoria no encontrados en el nivel superior.
- ***Tiempo de acierto*** (hit time). Tiempo necesario para determinar si el acceso es un acierto o un fallo.
- ***Castigo por fallo*** (miss penalty). Tiempo para reemplazar un bloque en el nivel superior con el correspondiente bloque del nivel inferior, mas el tiempo para entregar el bloque al procesador.

Memoria caché

Caché, es un lugar seguro para esconder o guardar cosas.

Nuevo diccionario Mundial del Lenguaje Americano

- **Memoria Caché** es el nivel de memoria situada entre el procesador y la memoria principal.
- Se comenzaron a usar a fines de los años 60s.
- Hoy en día, todas la computadoras incluyen cachés.

Usos de las memorias cachés

- ***Cachés de datos.*** Guardan los últimos datos referenciados.
- ***Cachés de instrucciones.*** Guardan las últimas instrucciones ejecutadas.
- ***Cachés de trazas*** (trace caches). Guardan secuencias de instrucciones para ejecutar que no son necesariamente adyacentes.

Ejemplo

- El tamaño del **bloque** es 1 palabra (4 bytes).
- Las peticiones de la CPU son de 1 palabra.
- La memoria caché ya tiene los siguientes datos:
- Se pide X_n , que no se encuentra en el caché.
- Entonces, se produce un fallo
- Se trae X_n de la memoria principal.
- Se inserta en el caché.

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_n
X_3

Dos Preguntas

1. ¿Cómo se sabe si un dato está en el caché?
 2. Si está, ¿Cómo se encuentra el dato?
- Estrategias que se usan:
 - a) **Caché de mapeo directo** (direct mapped cache).
 - b) **Caché asociativo total** (fully associative cache).
 - c) **Caché asociativo por conjunto** (n-way set associative cache).

Caché de mapeo directo

- A cada dato se le asigna un lugar en el caché de acuerdo a su dirección en la memoria principal.
- $b = d \bmod n$
 b es el bloque que le corresponde en el caché.
 d es la dirección de bloque en el caché.
 n es el número de bloques del caché.
- Si n es 2^m , el cache se puede indexar con los m bits mas bajos de su dirección de bloque.

Caché de mapeo directo

- $d = a \text{ div } k$

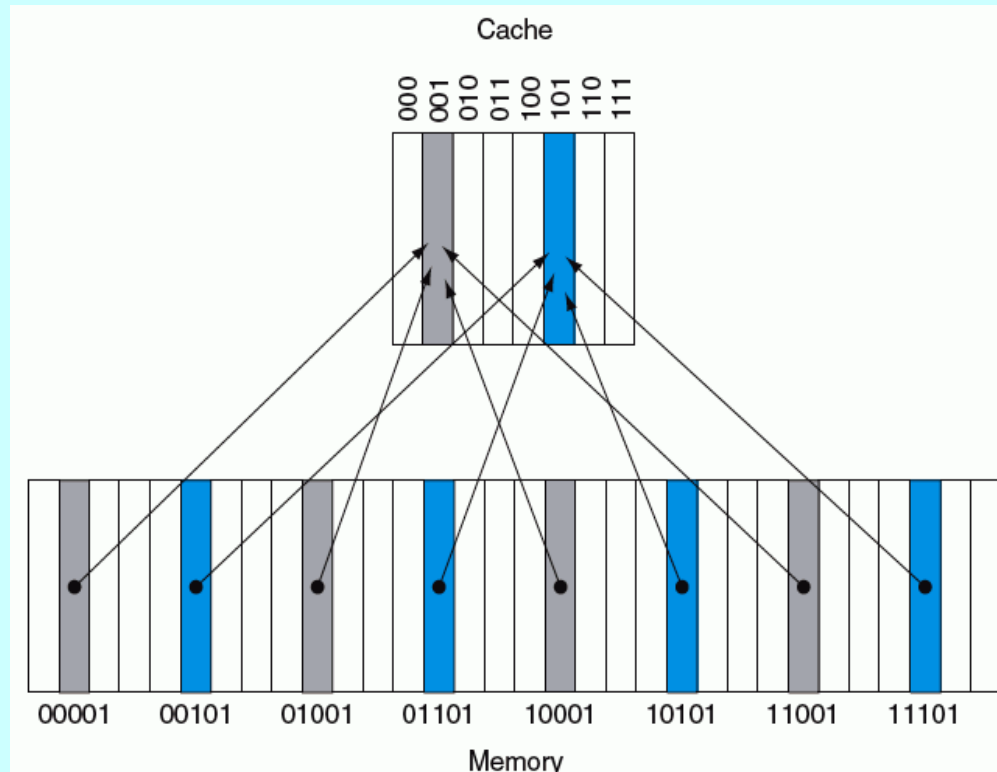
d es la dirección de bloque en el caché.

a es la dirección del dato en memoria.

k es el número de bytes por bloque.

Ejemplo

- Si n es 8, la dirección en el cache se encuentra con los 3 bits mas bajos de la dirección:



Etiquetas y bit válido

- Se necesita saber si el dato en el caché corresponde con el dato buscado.
- Cada bloque en el caché tiene una **etiqueta**.
- La **etiqueta** tiene la información necesaria para identificar si el dato en el caché es el dato buscado.
- En mapeo directo, la **etiqueta** tiene los bits altos de la dirección del dato.
- Además se necesita saber si el bloque tiene información válida o no.
- Cada bloque tiene un bit llamado **bit válido**.

Ejemplo

- Tamaño de la memoria caché: **8 bloques**.
- Tamaño del bloque: **1 palabra**.
- Inicialmente la memoria caché está vacía (para todas las entradas *bit válido* = *falso*).
- Se reciben las siguientes peticiones de direcciones de palabra: **22, 26, 22, 26, 16, 3, 16 y 18**

Estado inicial

Estado inicial de la memoria caché después de encender el computador:

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Se pide la dirección 22_{10}

- $22_{10} = 10110_2$.
- Se busca en el bloque 110 y se produce un fallo.
- Se carga el dato en el bloque 110.

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory(10110_{two})
111	N		

Se pide la dirección 26_{10}

- $26_{10} = 11010_2$.
- Se busca en el bloque 010 y se produce un fallo.
- Se carga el dato en el bloque 010.

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory(10110_{two})
111	N		

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Se pide la dirección 22_{10}

- $22_{10} = 10110_2$.
- Se busca en el bloque 110.
- Se compara la etiqueta con 10 (la parte alta de la dirección).
- Se produce un acierto.

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Se pide la dirección 26_{10}

- $26_{10} = 11010_2$.
- Se busca en el bloque 010.
- Se compara la etiqueta con 11 (la parte alta de la dirección).
- Se produce un acierto.

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Se pide la dirección 16_{10}

- $16_{10} = 10000_2$.
- Se busca en el bloque 000 y se produce un fallo.
- Se carga el dato en el bloque 000.

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Se pide la dirección 3_{10}

- $3_{10} = 00011_2$.
- Se busca en el bloque 011 y se produce un fallo.
- Se carga el dato en el bloque 011.

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Se pide la dirección 16_{10}

- $16_{10} = 10000_2$.
- Se busca en el bloque 000.
- Se compara la etiqueta con 10 (la parte alta de la dirección).
- Se produce un acierto.

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

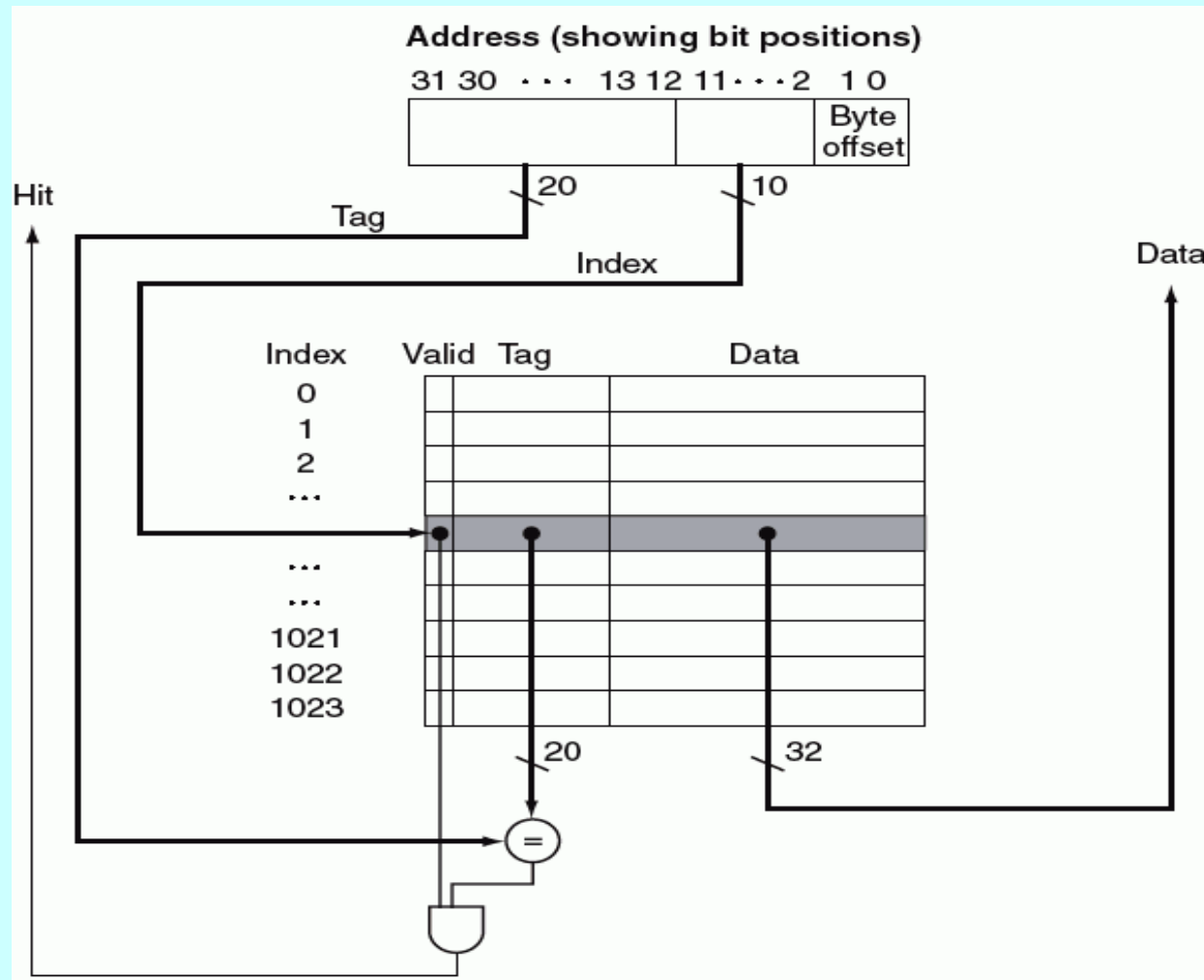
Se pide la dirección 18_{10}

- $18_{10} = 10010_2$.
- Se busca en el bloque 010 y se produce un fallo.
- Se carga el dato en el bloque 010.

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Caché de 4 kb de mapeo directo

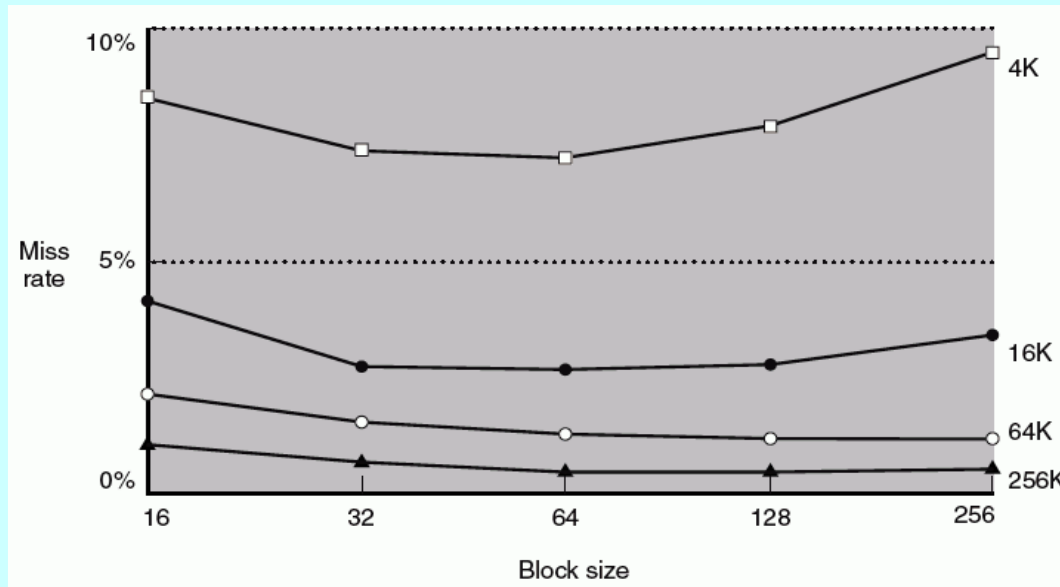


Explicación Diagrama anterior

- $1K = 1024 = 2^{10}$ palabras.
- Direcciones de 32 bits.
- La dirección de memoria principal se divide en:
 - **Índice del cache** (bits 2:11, **en total 10 bits**) que selecciona el bloque.
 - **Etiqueta** (bits 12:31, **en total 20 bits**) se compara con la etiqueta del bloque del caché.
- Acierto = Bit válido **AND** (*etiqueta* == dirección[12:31])
- Como en MIPS las palabras están alineadas en múltiplos de 4, los bits 0:1 que direccionan bytes se ignoran para seleccionar el bloque.

Tamaño del bloque

- Incrementar el tamaño del bloque generalmente reduce la tasa de fallos.
- La tasa de fallos aumenta si el bloque es demasiado grande comparado con el tamaño del cache.



Fallo del caché

- Una petición por datos (o instrucciones) que no se cumple porque el dato no está en el caché.
- Si el caché reporta acierto, la computadora usa el dato o instrucción y continúa como si nada.
- Si el caché reporta un fallo, el control debe obtener el dato del siguiente nivel de la jerarquía y posiblemente detener (stall) el pipeline.
- Los superescalares, con ejecución fuera de orden, pueden ejecutar otras instrucciones y *ocultar* el fallo.

Fallo de caché

- En un caché de instrucciones:
 1. Se envía el valor original del PC ($\text{PC actual} - 4$) a la memoria.
 2. Ordenar una lectura a la memoria y esperar a que se complete.
 3. Guardar la instrucción en el caché, escribiendo los bits altos de la dirección en la etiqueta y prendiendo el bit válido.
 4. Recomenzar la ejecución de la instrucción desde el primer paso.

Maapeo de escrituras

- Una escritura (***write***) es el resultado de un ***store***.
- El ***store*** debe escribir en el caché y en la memoria.
- De otro modo el caché y la memoria serían ***inconsistentes***.
- Dos estrategias se utilizan para el manejo de escrituras:
 - **Write-through**. Escribir cada vez en el caché y en la memoria.
 - **Write-back**. Escribir solo en el caché y copiar el dato a la memoria cuando la entrada en el caché va a ser remplazada.

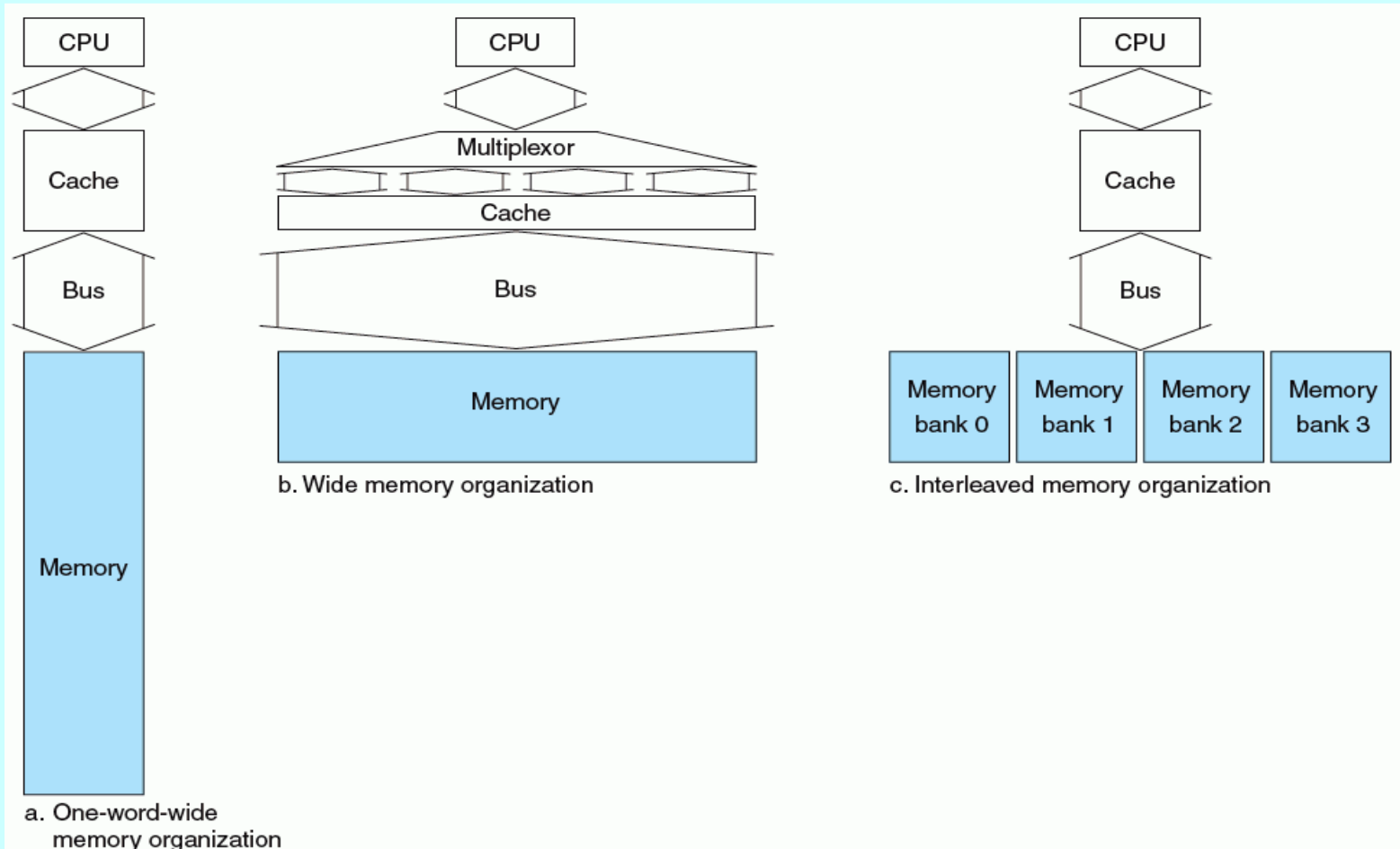
Comparación

- **Write-through** necesita un ciclo de reloj para escribir porque el dato simplemente se sobrescribe.
- **Write-back** necesita dos ciclos de reloj para escribir:
 - Un ciclo para ver si el dato está en el caché.
 - Otro ciclo para escribir el dato.
- Una solución es usar un buffer de store.
- El procesador busca en el caché y coloca el dato en el buffer durante el ciclo normal de acceso al caché.
- Suponiendo un acierto, el dato se escribe en el caché durante el siguiente ciclo sin usar.

Organización de la memoria

- **Objetivo:** reducir el castigo por fallo.
- La CPU está conectada a la memoria por un bus.
- El reloj del bus puede ser hasta 10 veces mas lento que el reloj del procesador.
- La velocidad del bus afecta al castigo por fallo.
- Suponer los siguientes tiempos:
 - 1 ciclo de reloj del bus para enviar la dirección.
 - 15 ciclos por cada acceso a memoria.
 - 1 ciclo para enviar una palabra.
- Suponer un tamaño de bloque de 4 palabras.

Organización de la memoria



Organización de la memoria

- En la organización de memoria llamada Ancho de 1 palabra:
 - Ancho del bus y de la memoria es de **1 palabra**.
 - El acceso a la memoria es secuencial.
 - Castigo por fallo: $1 + 4 \times 15 + 4 \times 1 = 65$ ciclos de reloj del bus.
 - Número de bytes transferidos por ciclo de reloj del bus por cada fallo: $(4 \times 4) / 65 = 0.25$

Organización de la memoria

- En la organización de memoria llamada **Memoria ancha**:
 - Ancho del bus y de la memoria es de **4 palabras**.
 - El acceso a la memoria es en paralelo.
 - Castigo por fallo: $1 + 1 \times 15 + 1 \times 1 = 17$ ciclos de reloj del bus.
 - Número de bytes transferidos por ciclo de reloj del bus por cada fallo: $(4 \times 4) / 17 = 0.94$

Organización de la memoria

- En la organización de memoria llamada *interleaved (entrelazado)*:
 - Ancho del bus es de **1 palabra**.
 - Ancho de la memoria es de 4 palabras repartidas en 4 bancos de memoria independientes.
 - El acceso a la memoria es en paralelo.
 - La transferencia es secuencial.
 - Castigo por fallo: $1 + 1 \times 15 + 1 \times 4 = 20$ ciclos de reloj del bus.
 - Número de bytes transferidos por ciclo de reloj del bus por cada fallo: $(4 \times 4) / 20 = 0.8$

Organización de la memoria

- La organización *interleaved(entrelazado)* es la preferida:
 - El castigo por fallo es aceptable.
 - Un bus y memoria de 4 palabras es mas caro que un bus y 4 bancos de 1 palabra cada uno.
 - Cada banco puede escribir en forma independiente, cuadruplicando el ancho de banda al escribir y provocando menos detenciones (*stalls*) en un caché **write-through**.

Resumen

- Caché de mapeo directo.
- Una palabra puede ir en un solo bloque y hay una etiqueta para cada bloque.
- Estrategias para mantener el caché y la memoria consistentes: **write-through** y **write-back**.
- Para tomar ventaja de la *locality espacial* el bloque del caché debe ser mayor a una palabra.

Resumen

- Bloques grandes reducen la tasa de fallos y mejoran la eficiencia al requerir menos espacio para las etiquetas.
- Bloques grandes incrementan el castigo por fallo.
- Para evitar el incremento se incrementa el ancho de banda de la memoria para transferir bloques mas eficientemente.
- Los dos métodos mas comunes para lograr el incremento son hacer la memoria mas ancha e *interleaving*.

Memoria Virtual

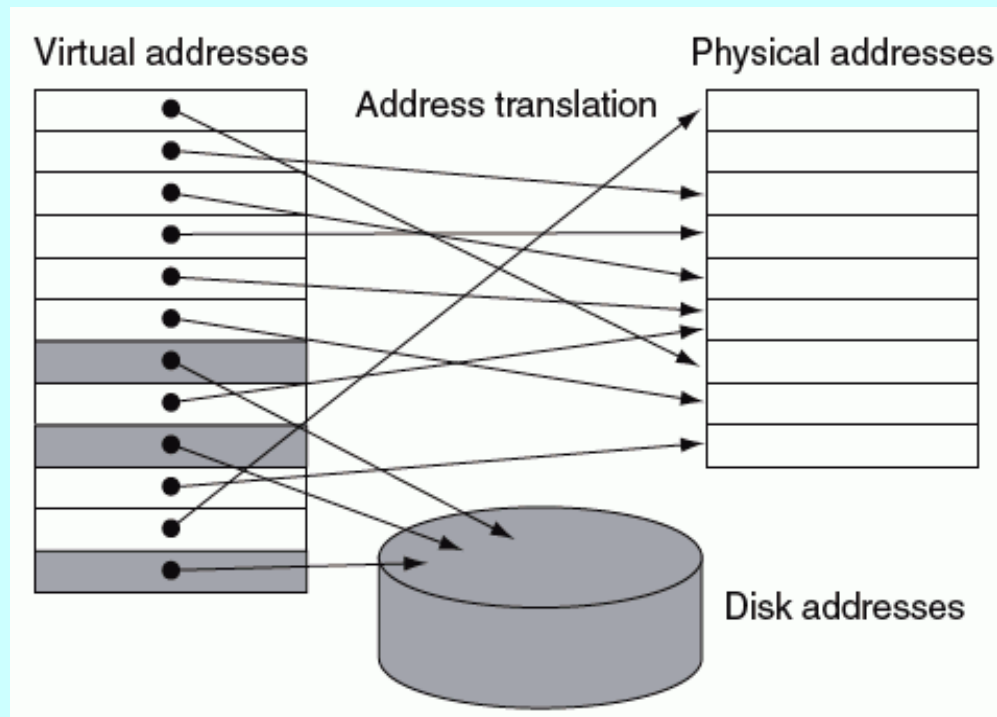
- La memoria principal actúa como caché de la memoria secundaria (disco duro). Esta técnica se denomina Memoria Virtual.
- 2 motivaciones principales:
 - Permitir que la memoria sea compartida en forma eficiente y segura por varios programas.
 - Eliminar los inconvenientes de programación asociadas a una memoria principal cuya capacidad sea pequeña.
- Memoria virtual es un mecanismo que permite que la memoria principal parezca mas grande que su tamaño físico.
- Permite ejecutar programas mas grandes que la memoria física disponible.

Memoria y cachés

- Los conceptos en memoria virtual y cachés son los mismos pero con nombres diferentes.
- Un **bloque** en memoria virtual se llama “**página**”.
- Un **fallo** en memoria virtual se llama “**fallo de página**”.
- En memoria virtual la CPU produce una “**dirección virtual**” que es traducida por hardware y software a una **dirección física** que se usa para acceder a la memoria principal.

Mapeo de direcciones

- Esta traducción se llama “*mapeo de direcciones*” o “*traducción de direcciones*”.

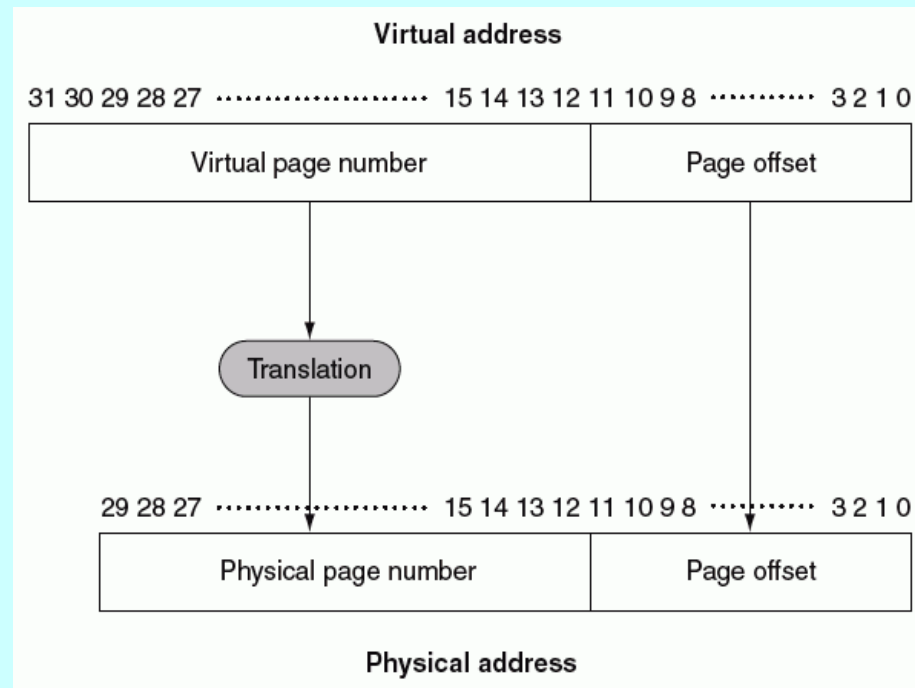


Explicación

- Hoy en día, los dos niveles de la jerarquía de la memoria controlados por la memoria virtual corresponden a las **DRAMs** y a los **discos magnéticos**.
- Según la analogía con la biblioteca, podemos pensar que una **dirección virtual** se corresponde con el título del libro; y que una **dirección física** se corresponde con su ubicación en la biblioteca donde se encuentra el libro, que le fue asignada según el código de identificación de la biblioteca.

Direcciones virtuales y físicas

- En memoria virtual, una dirección se compone de un número de página virtual y un offset.
- El número de página virtual se traduce a un número de página física.



Direcciones virtuales y físicas

- El número de bits del offset determina el tamaño de la página.
- El número de bits del número de página virtual es mayor o igual al número de bits del número de página física.
- La idea es que la memoria virtual sea mas grande que la memoria física.

Consideraciones de diseño

- El “*fallo de página*” es muy costosa.
- La memoria principal es **100,000 veces** más rápida que el disco duro.

Feature	Typical values for L1 caches	Typical values for L2 caches	Typical values for paged memory	Typical values for a TLB
Total size in blocks	250–2000	4000–250,000	16,000–250,000	16–512
Total size in kilobytes	16–64	500–8000	250,000–1,000,000,000	0.25–16
Block size in bytes	32–64	32–128	4000–64,000	4–32
Miss penalty in clocks	10–25	100–1000	10,000,000–100,000,000	10–1000
Miss rates (global for L2)	2%–5%	0.1%–2%	0.00001%–0.0001%	0.01%–2%

Consideraciones de diseño

- Las páginas deben ser suficientemente grandes para amortizar el costo de acceso. Los tamaños que se encuentran hoy en día van desde 4KB hasta los 16KB. Se están desarrollando nuevos sistemas que soportan páginas de 32KB y 64KB.
- Para reducir la **tasa de fallos**, los bloques pueden ir en cualquier parte de la memoria principal (i.e. la memoria es *fully associative*(totalmente asociativa)).
- *Los fallos de página* se manejan en el software para permitir algoritmos “inteligentes”.
- Cualquier reducción en la **tasa de fallos** vale la pena el esfuerzo de implementación.

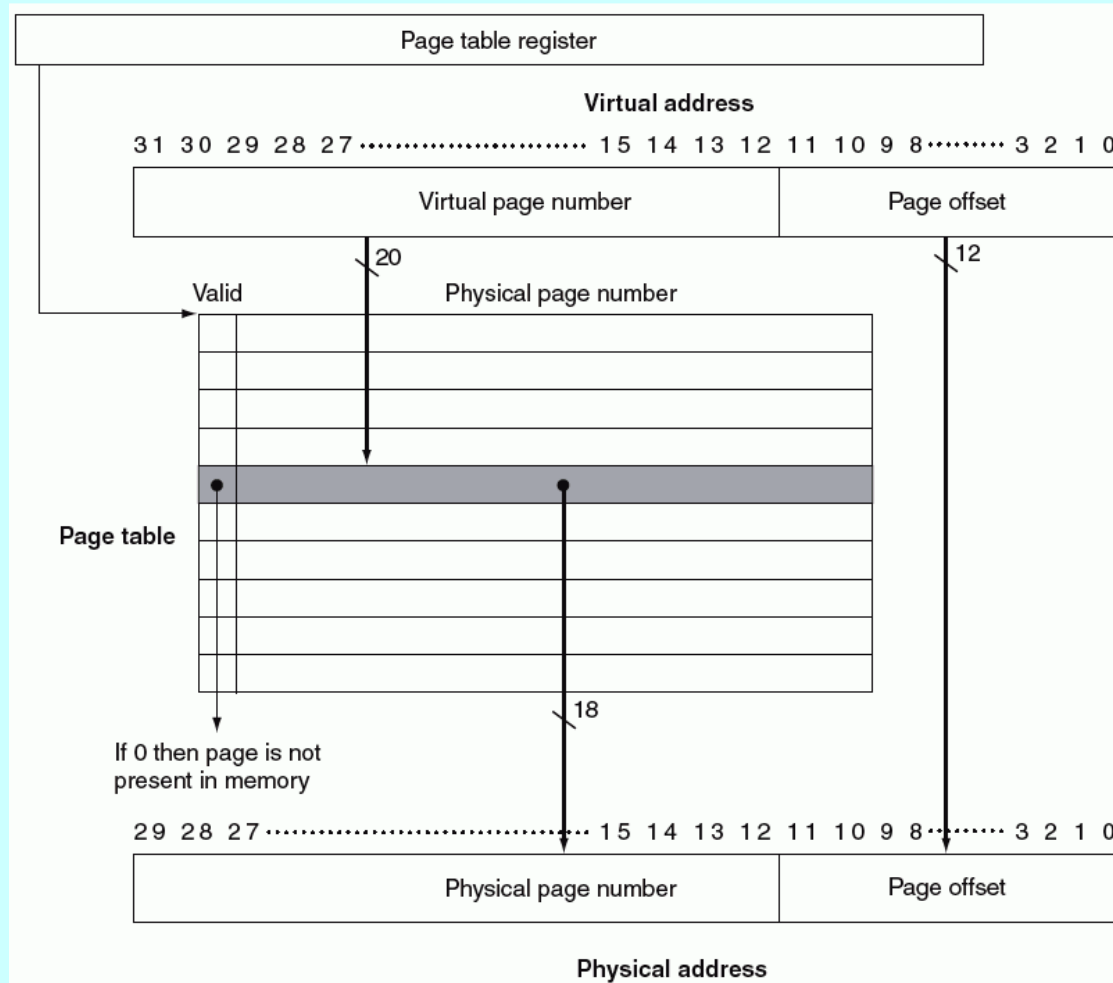
Consideraciones de diseño

- *Write-through* no funciona para memoria virtual. Los sistemas de memoria virtual usan *write-back*.

Buscando una página

- Una página puede estar en cualquier lugar de la memoria.
- Se usa una tabla para acceder a la memoria llamada “*tabla de páginas*”.
- La tabla de páginas se indexa con el número de página virtual y regresa el número de página física.
- La CPU incluye un registro para apuntar a la tabla de páginas del programa que está corriendo.

Tabla de páginas



Explicación

- El registro de tabla de páginas apunta al comienzo de la tabla de páginas.
- El tamaño de página es $2^{12} = 4$ KB.
- El espacio de direcciones virtuales es $2^{32} = 4$ GB.
- El espacio de direcciones físicas es $2^{30} = 1$ GB.
- El número de entradas en la tabla de páginas es $2^{20} = 1,048,576$.
- Cada entrada tiene un bit válido para indicar si el mapeo es legal.
- Si el bit es falso, se genera un *fallo de página*.

Tabla de Páginas

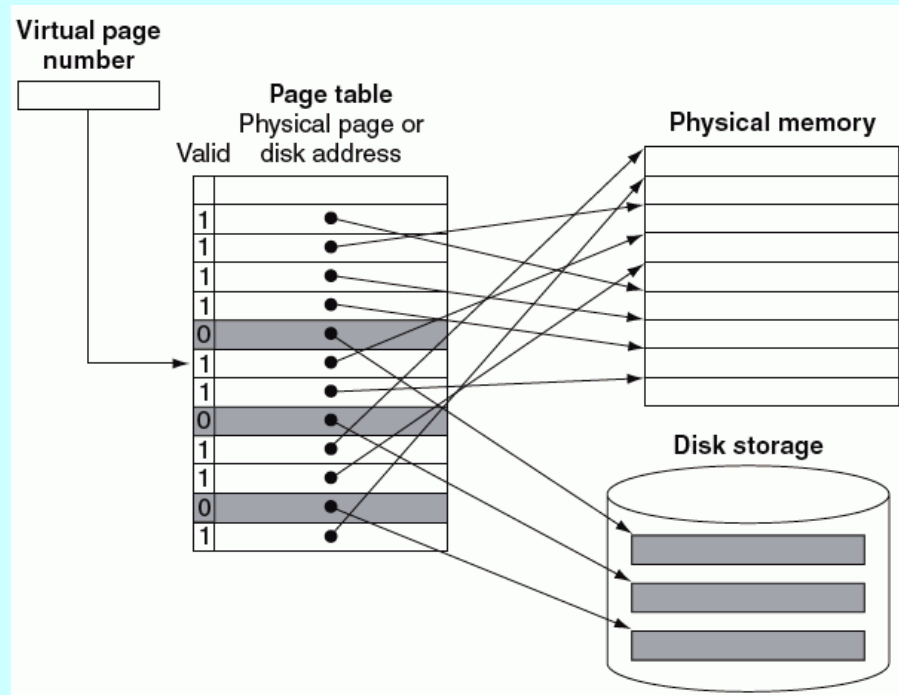
- Cada programa tiene su propia tabla de páginas.
- Varios programas pueden tener el mismo espacio de direcciones virtuales.
- El sistema operativo se encarga de:
 - *Asignar la memoria física.*
 - *Actualizar las tablas de páginas para que los espacios de direcciones virtuales de los distintos programas no colisionen.*

Fallo de página

- Ocurre cuando el bit válido es falso.
- El sistema operativo:
 - Toma el control.
 - Interrumpe el proceso y guarda su estado.
 - Busca la página en el **área de swap** del disco duro y la carga en la memoria principal.
 - Si la memoria está llena, se necesita reemplazar una página.

Área de swap

- **Área del disco reservada** para todo el espacio de memoria virtual de un proceso.
- Incluye un registro de donde se guarda en disco cada página virtual.



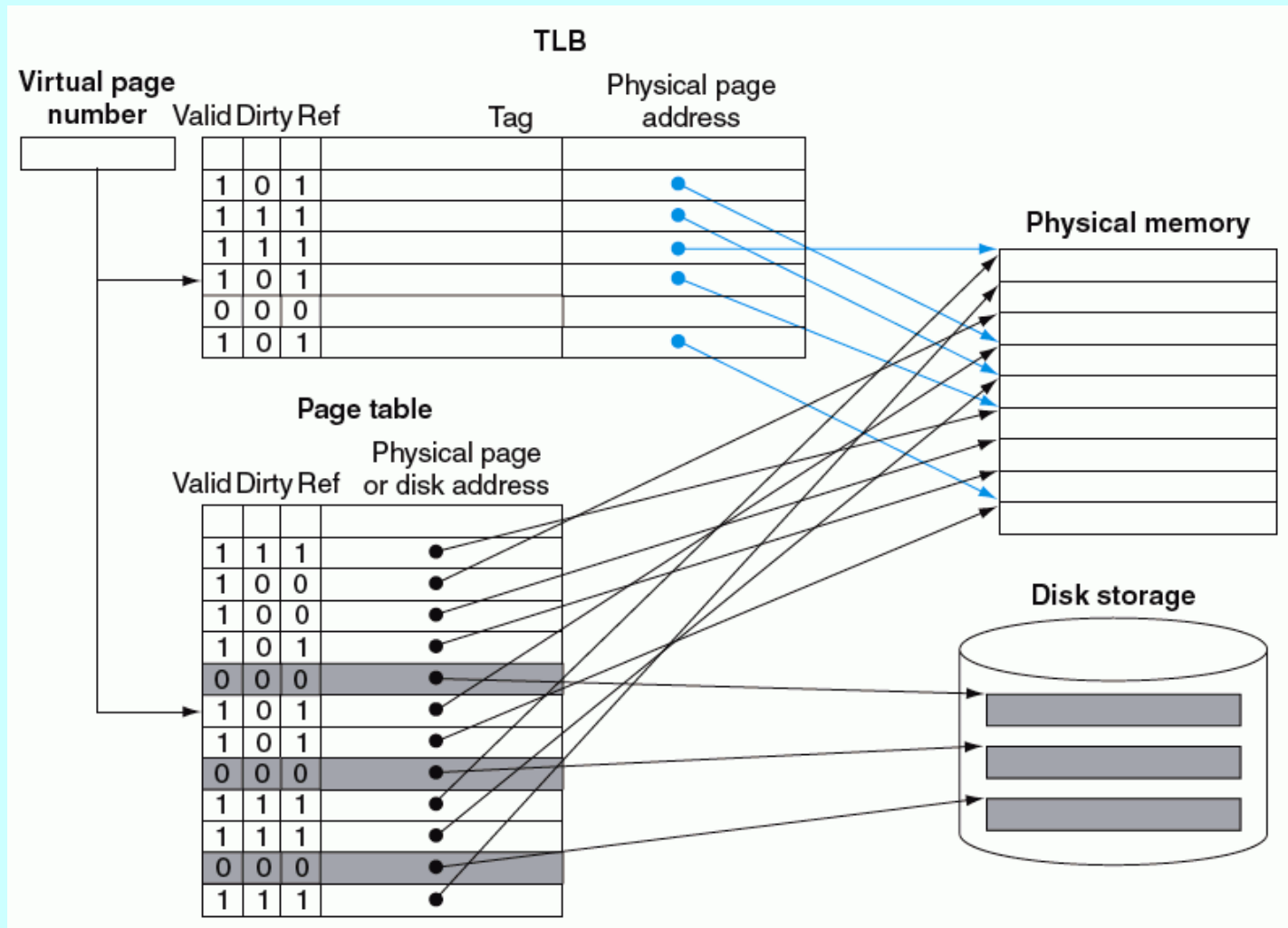
Tamaño de la tabla de páginas

- Suponer lo siguiente:
 - Direcciones virtuales de 32 bits.
 - Número de página virtual ocupa 20 bits y el offset los otros 12 bits.
- El número de entradas en la tabla de páginas es $2^{20} = 1,048,576$.
- Si cada entrada ocupa 4 bytes, la tabla ocupa 4 MB.
- Si hay cientos de procesos, la memoria puede ser insuficiente.
- Es deseable reducir el tamaño de la tabla.

TLB

- Aunque *las tablas de páginas* residan en memoria principal, cada acceso toma el doble de tiempo:
 - Un acceso para obtener la dirección física.
 - Un segundo acceso para obtener los datos.
- Para mejorar el rendimiento se aprovecha el concepto de “locality”.
- Si una traducción de una dirección virtual se acaba de usar, es posible que se vuelva a usar.
- Se usa un caché especial de traducciones llamado **TLB** (*translation lookaside buffer*)(búfer de traducción de páginas). Es la caché de la tabla de páginas.

TLB



Explicación de TLB

- La **TLB** actúa como caché para las entradas que mapean solo a páginas físicas.
- La **TLB** contiene un subconjunto de los mapeos de página virtual a página física que están en la tabla de páginas.
- Como el **TLB** es un caché tiene campo de etiqueta.
- Si la página no está en el **TLB** se busca en la tabla de páginas.

Explicación del TLB

- Se busca la página virtual en la tabla de páginas y sucede una de dos cosas:
 - a) Regresa el número de página física de la página (que se usa para construir una entrada en la TLB).
 - b) Indica que la página reside en disco, en cuyo caso se genera un fallo de página.
- La tabla de páginas no tiene campo de etiqueta porque **no** es un cache.